

## 5. Ingeniería de Modelos y Optimización para TinyML

El presente capítulo aborda la fase de Ingeniería de Modelos, cuyo propósito central es el entrenamiento, evaluación y optimización algorítmica de arquitecturas de aprendizaje profundo (Deep Learning) adaptadas para la detección de objetos en microcontroladores de recursos estrictamente limitados. Partiendo del conjunto de datos egocéntrico curado, balanceado y redimensionado en el capítulo anterior, se establece un ecosistema de experimentación riguroso para evaluar el rendimiento de modelos ligeros.

Para cumplir con las restricciones del microcontrolador ESP32-S3 y la cámara OV5640, la investigación se centra en arquitecturas de alta eficiencia computacional, destacando implementaciones basadas en YOLO (variantes nano como *YOLO26n*), redes anchor-free especializadas para ESP32 (*ESPDet*) y topologías soportadas por extractores de características como *MobileNetV3* con cabezas tipo FCOS (*Fully Convolutional One-Stage*). El objetivo es consolidar modelos que no solo garanticen una alta precisión media promedio (*mAP*), sino que mantengan un tamaño binario inferior a 3.0 MB y latencias viables para la navegación en robótica móvil de bajo coste.

### 5.1. Infraestructura y Entorno de Entrenamiento Remoto

El procesamiento de redes neuronales convolucionales complejas y la gestión de grandes volúmenes de datos exigieron una infraestructura capaz de garantizar la reproductibilidad, trazabilidad y escalabilidad de los experimentos. En este sentido, se diseñó un flujo de trabajo basado en el paradigma de Operaciones de Aprendizaje Automático (MLOps), el cual permitió separar lógicamente el entorno de desarrollo local de los nodos de ejecución intensiva en la nube.

#### 5.1.1. Evolución Metodológica

Para mitigar riesgos técnicos y consolidar el dominio sobre las herramientas y Frameworks de entrenamiento implementados, tanto a nivel de la estación de trabajo local como en la estación de trabajo MLOps implementada en *Google Vertex AI*, la orquestación se ejecutó en tres fases evolutivas:

- **Ciclo 0 - Experimentación de Modelos en Local:** Antes de consolidar el estudio, se evaluaron tres paradigmas de detección para identificar el equilibrio óptimo entre precisión y huella de memoria en el SoC ESP32-S3 (N16R8). El proceso de experimentación preliminar funcionó como un filtro técnico; se crearon dos notebooks de llamados: ``03_TrainModels.ipynb`` y ``04_TrainModels_2.ipynb`` donde se realizaron pruebas con distintos modelos iniciales (*MobileNetV2*, *MobileNetV3 + SSD-Lite*, *FOMO from scratch*, *YOLOv11*, *YOLO26*) para reconocimiento de metodología, ventajas, desventajas, métricas, entre otros aspectos generales. Todo lo necesario para refinar los pipelines de experimentación en el ciclo 1 y la elección de los modelos base a comparar. Los modelos finales se eligieron basándose en su capacidad para operar bajo la restricción de memoria de ~3.0 MB (con un objetivo operativo de ~2.0 MB) y el aprovechamiento de las instrucciones vectoriales del procesador.
- **Ciclo 1 – Experimentación y Diagnóstico de MLOps:** Esta fase validó la viabilidad del pipeline MLOps y la comunicación entre los repositorios de *Google Cloud Storage* (GCS) y los nodos de cómputo en la estación local. A través de pruebas de entrenamientos de modelos como *MobileNetV3 + SSD-Lite*, *YOLOv11* y *YOLO26* se identificaron cuellos de botella críticos, tales como errores de desbordamiento de memoria (OOM, *Out Of Memory*) al gestionar lotes de entrenamiento (batch sizes) grandes y conflictos de dependencia entre *frameworks*. Estos hallazgos permitieron ajustar los requerimientos y la arquitectura de recursos antes de la fase final.
- **Ciclo 2 – Entrenamiento definitivo y Optimización:** Con el entorno estabilizado a partir del diagnóstico del Ciclo 1, se procedió a la ejecución formal de los entrenamientos. En esta fase se implementaron las estrategias de *Transfer Learning*, *Fine-tuning* y entrenamiento multiescala definitivas. Se consolidó el uso de contenedores *Docker* específicos para cada familia de modelos y se automatizó la persistencia de los artefactos resultantes (pesos, métricas y topologías cuantizables).

#### 5.1.2. Configuración y organización de ficheros

Para mantener un enfoque profesional y organizado en la ingeniería de modelos, se produjeron ficheros adicionales que soportan la modularización y funcionamiento de los notebooks como entornos de experimentación; estos ficheros son:

- **Scripts:** con modularización de clases y funciones utilizadas en pipelines, organizados en ficheros ``src_`` y ``scripts``
- **Datasets:** convertidos a formatos apropiados para los dos modelos seleccionados, organizados en el fichero ``datasets``
- **Logs:** con registros de entrenamientos de los experimentos, organizados en el fichero ``logs``
- **Modelos:** .keras y .pt obtenidos en checkpoints y versiones finales de cada modelo entrenado en cada experimento, organizados en el fichero ``models``
- **Reportes:** de figuras y métricas obtenidas en cada experimento realizado, organizados en el fichero ``reports``.

### 5.1.3. Arquitectura y Configuración de Nodos en GCS

La arquitectura final implementada para el Ciclo 2, ilustrada en la Figura 33, separa lógicamente el entorno de desarrollo local (estación de trabajo) del nodo de ejecución en la nube.

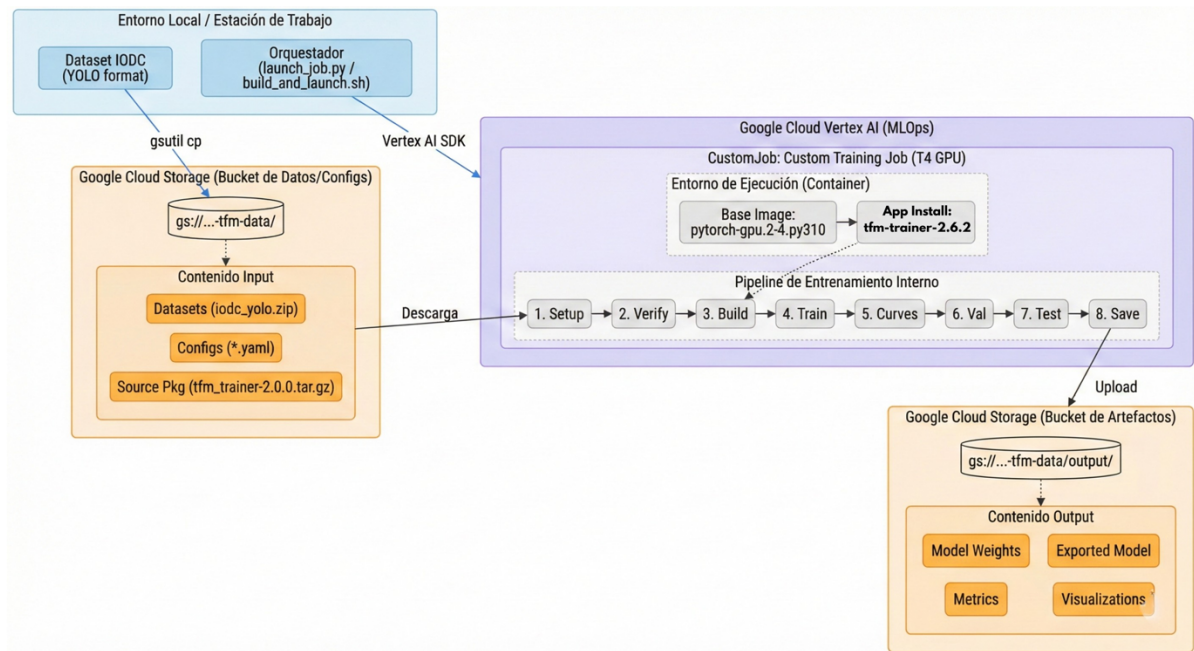
La orquestación se centralizó a través de trabajos personalizados (*Custom Jobs*) en *Vertex AI*. Para mantener un equilibrio óptimo entre capacidad de procesamiento paralelo y costos operativos, cada experimento instanció una máquina virtual tipo **n1-standard-8** (equipada con 8 vCPUs y 30 GB de memoria RAM), acelerada por hardware mediante una **GPU NVIDIA Tesla T4**. Esta asignación fue fundamental para prevenir errores de memoria durante la aplicación de técnicas de aumento de datos en línea (*data augmentation*) agresivas durante los entrenamientos.

Para garantizar el aislamiento de procesos y la compatibilidad de librerías, el software se contenerizó mediante imágenes *Docker* oficiales preconfiguradas para aceleración por *GPU*, adaptándose a los requerimientos de cada arquitectura:

- **Ecosistema PyTorch (*pytorch-gpu.2-4.py310*):** Utilizado para el entrenamiento de la familia *YOLO* (*YOLO11*, *YOLO26*), redes anchor-free como *FCOS* y redes ligeras como *ESPDet*. Estos modelos consumieron los datos directamente en formato de anotaciones *YOLO*.

- **Ecosistema TensorFlow/Keras (*tf-gpu.2-17.py310*):** Empleado para las topologías basadas en MobileNet (V2 y V3). El procesamiento de datos se realizó de forma paralelizada mediante el estándar *TFRecord*.

**Figura 33.** Mapa Espacial de Objetos Antes y Después de Aumentación



#### 5.1.4. Flujo de Datos

La lógica de experimentación se centralizó en un directorio de control modular (*Train\_MLOps/*), el cual separó los scripts de empaquetado (*setup.py*), la gestión de dependencias y los puntos de entrada para los trabajos remotos (*trainer/*). El flujo de datos sigue un modelo de fuente única de verdad en la nube, donde GCS actúa como el núcleo del sistema.

Como lo detalla las Figuras 33, 34 y 35, el ciclo 2 de entrenamientos optimizó la orquestación, de tal modo que el flujo principal era el siguiente:

- **Ingesta y Preparación Local:** Los datos (*Dataset IODC*) se organizan localmente en formato *YOLO*, se comprimen en un archivo *.zip* para optimizar la transferencia y se suben a un contenedor (*bucket*) de GCS (*gs://project-...-tfm-data/datasets/*).
- **Suministro de Configuración:** Los hiperparámetros y la definición de los modelos no están grabados (*hardcoded*) en el script de entrenamiento. Se utilizan archivos *YAML*



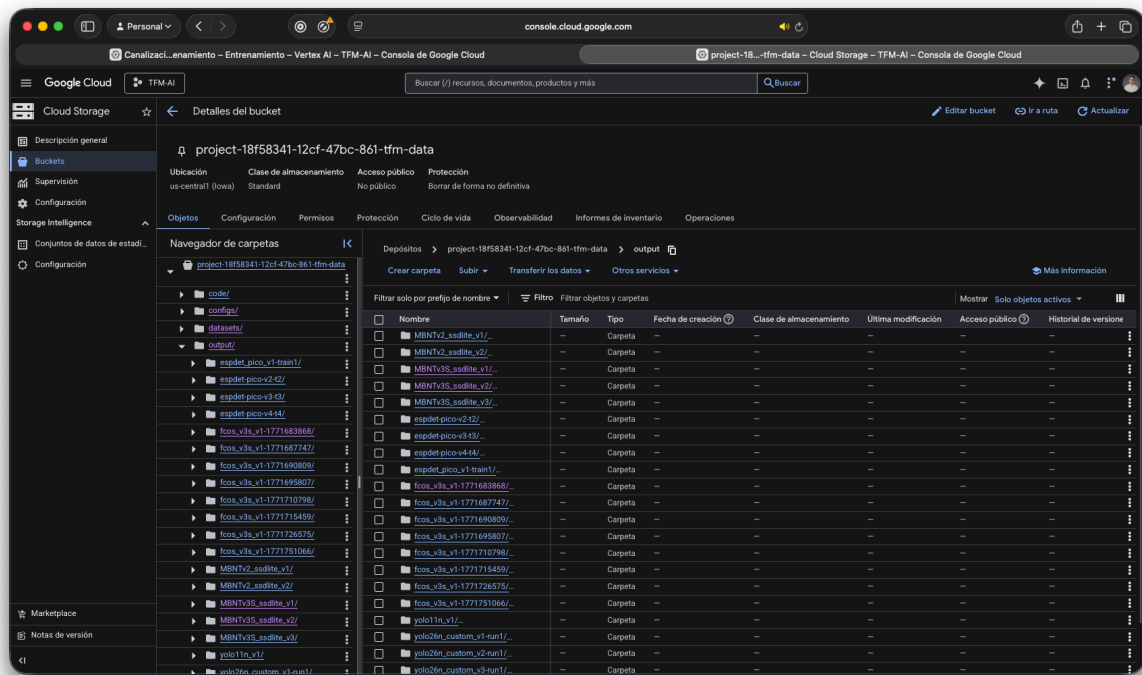
independientes que se cargan a GCS, permitiendo versionar los experimentos sin cambiar el código base.

- **Transferencia al Nodo de Cómputo:** Al lanzar un *Custom Job* en *Vertex AI*, el sistema descarga automáticamente tres elementos desde *GCS* al contenedor: el *dataset* comprimido, la configuración del experimento (YAML) y el código de entrenamiento empaquetado (*tfm\_trainer-2.X.X.tar.gz*).
- **Seguimiento y Telemetría:** La convergencia de los modelos y las métricas de evaluación (tales como funciones de pérdida y mAP) se auditan continuamente mediante logs JSON y herramientas de telemetría de *Vertex AI*. Con este monitoreo en tiempo real se diagnostican anomalías y se audita el consumo de VRAM y CPU.
- **Persistencia de Resultados:** Una vez finalizado el entrenamiento, los artefactos (modelo .pt, exportación ONNX, curvas de aprendizaje y métricas de evaluación) se suben de nuevo al *bucket* en una carpeta estructurada por el nombre de la versión del experimento (*run*). Esto asegura que cada modelo de cada arquitectura quede disponible para la posterior etapa de evaluación y cuantización a enteros de 8 bits (INT8).

**Figura 34.** Servicio de Entrenamientos en Google Cloud Vertex AI

| Nombre                           | ID                  | Estado      | Tipo de trabajo               | Tipo de modelo | Duration    | Última actualización       | Fecha de creación          | Finalizada                 | Etiquetas |
|----------------------------------|---------------------|-------------|-------------------------------|----------------|-------------|----------------------------|----------------------------|----------------------------|-----------|
| tfm-yolo26n_custom_v3            | 6267808829191225344 | Finalizado  | Canalización de entrenamiento | Personalizado  | 49 min 48 s | 24 feb 2026, 7:22:34 p.m.  | 24 feb 2026, 6:32:37 p.m.  | 24 feb 2026, 7:22:34 p.m.  | —         |
| tfm-espdet_pico_v4-1771901256    | 7265276981816590336 | Finalizado  | Canalización de entrenamiento | Personalizado  | 27 min 17 s | 23 feb 2026, 10:14:57 p.m. | 23 feb 2026, 9:43:39 p.m.  | 23 feb 2026, 10:14:57 p.m. | —         |
| tfm-espdet_pico_v3-1771894463    | 57940186008378368   | Finalizado  | Canalización de entrenamiento | Personalizado  | 33 min 29 s | 23 feb 2026, 9:01:18 p.m.  | 23 feb 2026, 8:27:45 p.m.  | 23 feb 2026, 9:01:18 p.m.  | —         |
| tfm-espdet_pico_v2-1771891782    | 1216450101048770560 | Finalizado  | Canalización de entrenamiento | Personalizado  | 28 min 11 s | 23 feb 2026, 7:38:27 p.m.  | 23 feb 2026, 7:09:44 p.m.  | 23 feb 2026, 7:38:27 p.m.  | —         |
| tfm-espdet_pico_v2-1771890482    | 8560413718406496256 | Con errores | Canalización de entrenamiento | Personalizado  | 6 min 34 s  | 23 feb 2026, 6:54:39 p.m.  | 23 feb 2026, 6:48:05 p.m.  | 23 feb 2026, 6:54:39 p.m.  | —         |
| tfm-espdet_pico_v1-1771880828    | 1278128305320493056 | Finalizado  | Canalización de entrenamiento | Personalizado  | 30 min 44 s | 23 feb 2026, 4:37:59 p.m.  | 23 feb 2026, 4:07:11 p.m.  | 23 feb 2026, 4:37:59 p.m.  | —         |
| tfm-yolo26n_custom_v2-1771774142 | 7775077098732486656 | Finalizado  | Canalización de entrenamiento | Personalizado  | 40 min 24 s | 22 feb 2026, 11:09:34 a.m. | 22 feb 2026, 10:29:10 a.m. | 22 feb 2026, 11:09:34 a.m. | —         |
| tfm-yolo26n_custom_v1-1771760890 | 4911822920096940032 | Finalizado  | Canalización de entrenamiento | Personalizado  | 33 min 17 s | 22 feb 2026, 7:21:32 a.m.  | 22 feb 2026, 6:48:13 a.m.  | 22 feb 2026, 7:21:32 a.m.  | —         |
| tfm-fcos_v3a_v1-1771751066       | 1648613146346127360 | Finalizado  | Canalización de entrenamiento | Personalizado  | 23 min 4 s  | 22 feb 2026, 4:27:40 a.m.  | 22 feb 2026, 4:04:29 a.m.  | 22 feb 2026, 4:27:40 a.m.  | —         |
| tfm-fcos_v3a_v1-1771749400       | 7810355473240031232 | Con errores | Canalización de entrenamiento | Personalizado  | 6 min 19 s  | 22 feb 2026, 3:43:04 a.m.  | 22 feb 2026, 3:36:44 a.m.  | 22 feb 2026, 3:43:04 a.m.  | —         |
| tfm-fcos_v2a_v1-1771748886       | 307259505352536064  | Detenido    | Canalización de entrenamiento | Personalizado  | 5 min 58 s  | 22 feb 2026, 3:34:08 a.m.  | 22 feb 2026, 3:28:09 a.m.  | 22 feb 2026, 3:34:08 a.m.  | —         |
| tfm-fcos_v3a_v1-1771726575       | 3296024612382441472 | Finalizado  | Canalización de entrenamiento | Personalizado  | 29 min 18 s | 21 feb 2026, 9:45:56 p.m.  | 21 feb 2026, 9:16:37 p.m.  | 21 feb 2026, 9:45:56 p.m.  | —         |

**Figura 35.** Servicio de Contenedores en Google Cloud Storage (GCS)



### 5.1.5. Orquestación

La orquestación se divide en el control externo (lanzamiento) y el interno (ejecución del *Custom Job*). En la orquestación externa se implementaron dos mecanismos para disparar los trabajos en la nube:

- **Scripts de Bash (*build\_and\_launch.sh*):** Automatizan el ciclo completo de empaquetar el código Python, subirlo a GCS y ejecutar el comando de lanzamiento.
- **Python Launcher (*launch\_job.py*):** Utiliza el SDK de *Vertex AI* para definir la infraestructura necesaria: tipo de máquina (*n1-standard-8*), acelerador (*NVIDIA\_TESLA\_T4*) y la imagen del contenedor de *PyTorch*.

La orquestación interna, que contempla el ciclo de vida del *Custom Job*, dentro del contenedor de Vertex AI se divide en 8 bloques lógicos que aseguran la trazabilidad de cada experimento:

- **Configuración (*Setup*):** Preparación del entorno.
- **Verificación (*Verify*):** Validación de datos y configuraciones.
- **Construcción (*Build*):** Construcción del modelo basado en el YAML.
- **Entrenamiento (*Train*):** Entrenamiento del modelo con la GPU T4.
- **Gráficas (*Curves*):** Generación de gráficas de pérdida y precisión.

- **Evaluación (Val/Test):** Evaluación en conjuntos de datos independientes.
- **Almacenamiento (Save + Upload):** Empaquetado y subida final a GCS.

La orquestación tubo mejoras en el ciclo 2 de entrenamientos con respecto al ciclo 1, especialmente en la escalabilidad (lanzar entrenamientos en paralelo), gestión de errores (se incluyó parche para evitar entrenamientos distribuidos) y eficiencia de costos (lanzamiento opcional de *Custom Jobs* sin GPU para reducir gastos operativos).

## 5.2. Metodología de Entrenamiento

Dada la arquitectura de recursos limitados del microcontrolador ESP32-S3 y la necesidad de maximizar la precisión en un *dataset* especializado de asistencia visual, se determinó que el entrenamiento desde cero (*from scratch*) resultaría ineficiente debido a la convergencia lenta y al riesgo de sobreajuste por el volumen del *dataset*. En su lugar, se implementó una estrategia de optimización basada en un enfoque secuencial de transferencia de conocimiento, diseñado para adaptar extractores de características robustos a las particularidades del entorno de robótica móvil de bajo costo.

### 5.2.1. Estrategia de Aprendizaje

La estrategia se estructuró en dos etapas diferenciadas, Transfer Learning y Fine-Tuning, para asegurar la estabilidad de los gradientes durante el proceso de aprendizaje:

- **Fase 1 - Warm-up y Congelamiento del Backbone (Épocas 0 - 50):** Los entrenamientos se iniciaron utilizando pesos pre-entrenados para asegurar una base de conocimiento visual sólida. El Modelo 1 (FCOS) empleó pesos de ImageNet, el Modelo 2 (YOLO26n) partió de un checkpoint de COCO, y el Modelo 3 (ESPDet-Pico) utilizó una inyección de conocimiento específica mediante un modelo de detección disponible en el repositorio oficial de Espressif (cat-detection), logrando una transferencia del 99.97% de los parámetros. Durante esta fase, se congelaron las capas de extracción de características (*backbone*), permitiendo que únicamente las cabezas de detección aprendieran a mapear las clases específicas a detectar: *dog, person, door, stair y obstacle*.
- **Fase 2 - Descongelamiento y Ajuste Fino (Épocas 51 - 150):** Se procedió a la liberación de todos los parámetros de la red (*unfreeze*). Para evitar la degradación de las

representaciones visuales previamente adquiridas, se redujo significativamente la tasa de aprendizaje. Esta transición se realizó con el objetivo de lograr una convergencia suave, donde el modelo refina sus filtros convolucionales para detectar bordes y texturas críticas en condiciones de iluminación variable, típicas de los entornos interiores.

### 5.2.2. Optimización Algorítmica

La optimización algorítmica se centró en la robustez frente al desbalance de clases y la eficiencia computacional, integrando:

- **Optimizadores y Learning Rate ( $lr$ ):** Se seleccionó el optimizador *AdamW* para los modelos FCOS y ESPDet-Pico por su eficiente manejo del decaimiento de pesos. Para el modelo YOLO26n se usa el optimizador MuSGD, un nuevo optimizador híbrido que fusiona lo mejor de *SGD* con *Moun*. Se aplicó una política de tasas diferenciadas: una tasa agresiva de  $1 \times 10^{-3}$  durante el congelamiento y una tasa reducida de  $1 \times 10^{-4}$  durante el ajuste fino integral.
- **Funciones de Pérdida (*Loss Functions*):** Para contrarrestar la diversidad visual de la clase obstacle y mejorar la precisión, se integró el uso de funciones de pérdida que penalizan los falsos positivos en el fondo.
- **Parada temprana (*Early Stopping*) y Control de Sobreescritura (*Overfitting*):** Con el fin de optimizar el tiempo de cómputo en la infraestructura de *Vertex AI*, se configuró una política de *Early Stopping* con una paciencia de 25 épocas para detener el entrenamiento una vez que la métrica de validación (mAP) alcanzó su punto de saturación, evitando que los modelos se especializaran excesivamente en el ruido del conjunto de entrenamiento.

### 5.2.3. Entrenamiento Multiescala (Multi-scale Training)

Un componente vital de la metodología fue el uso de entrenamiento multiescala manual (*Progressive Resizing*). Durante el ciclo de entrenamiento, la resolución de entrada se varió dinámicamente, iniciando en 640x640 píxeles para permitir que el modelo aprendiera características de alta resolución, y reduciéndola escalonadamente a 416x416 y 320x320 píxeles. Este proceso culminó con una fase de estabilización rígidamente fijada en 224x224 píxeles a partir de la época 70. Esta resolución es la meta operativa para la inferencia en

tiempo real mediante la cámara OV5640 en el ESP32-S3. Con esto se buscó que se especialicen las estadísticas de normalización de la red para manejar los artefactos y la pérdida de detalle inherentes a la baja resolución del sensor final.

### 5.3.Desarrollo y Entrenamiento de Arquitecturas

#### 5.3.1. Modelo 1: Arquitectura MobileNetV3-Small + FCOS

Esta arquitectura se diseñó bajo la filosofía de eficiencia extrema para su ejecución en sistemas embebidos. El diseño compone tres bloques fundamentales:

- **Backbone:** Se utilizó *MobileNetV3-Small* pre-entrenado con *ImageNet*. Se extrajeron mapas de características en tres niveles de resolución: *stage 3 (stride 8)*, *stage 4 (stride 16)* y *stage 5 (stride 32)*.
- **Neck (Lightweight FPN):** Para la fusión de características multiescala, se implementó una *Feature Pyramid Network* (FPN) simplificada a 64 canales. Se emplearon convoluciones de  $1 \times 1$  para la reducción de dimensionalidad y bloques de suavizado basados en *Depthwise Separable Convolutions* (DSConv) para minimizar el uso de memoria PSRAM.
- **Head (FCOS Anchor-free):** La cabeza de detección opera por nivel de FPN y consta de tres ramas: clasificación para las 5 clases objetivo, regresión de distancias (l, t, r, b) y una rama de Centerness para suprimir predicciones descentradas de baja calidad.

Los entrenamientos se rigieron por el enfoque descrito en la sección 5.2. y sus resultados se detallan en la Tabla 13 y 14. Se realizaron ocho fases de experimentación incremental para identificar el balance óptimo entre precisión y recuperación (recall).

- **Establecimiento del Baseline (Train 1):** Se identificó un desbalance crítico donde la pérdida de regresión dominaba a la de clasificación debido al uso de píxeles absolutos.
- **Normalización por Stride (Train 2):** Se implementó la normalización de los targets de regresión según el stride de la FPN. Este ajuste fue el más significativo de la serie, incrementando el *mAP@50* en un 30%.

- **Integración de GloU Loss (Train 3):** Se substituyó *Smooth L1* por *Generalized IoU* (GloU) para optimizar directamente la métrica de intersección. Se logró una mejora en la calidad geométrica de las cajas (mAP@50-95 de 0.26).
- **Scoring y Calibración (Train 4 & 7):** Se probaron funciones de puntuación *IoU-aware*. Aunque aumentaron el mAP@50 a un máximo de 0.6120, dispararon los Falsos Positivos (FP), reduciendo la viabilidad operativa.
- **Aumentación de Datos (Train 5 & 6):** La inclusión de ruido gaussiano y *coarse dropout* degradó la precisión del modelo pequeño, sugiriendo que las arquitecturas ligeras son sensibles a la destrucción de información en datasets reducidos.
- **Análisis de Focal Loss (Train 8):** Se evaluó *Focal Loss* ( $\gamma=3.0$ ). Los resultados demostraron un colapso en la precisión (0.21). Se concluye que valores altos de  $\gamma$  sobre-penalizan los ejemplos positivos en modelos de baja capacidad, comprimiendo excesivamente los scores de confianza.

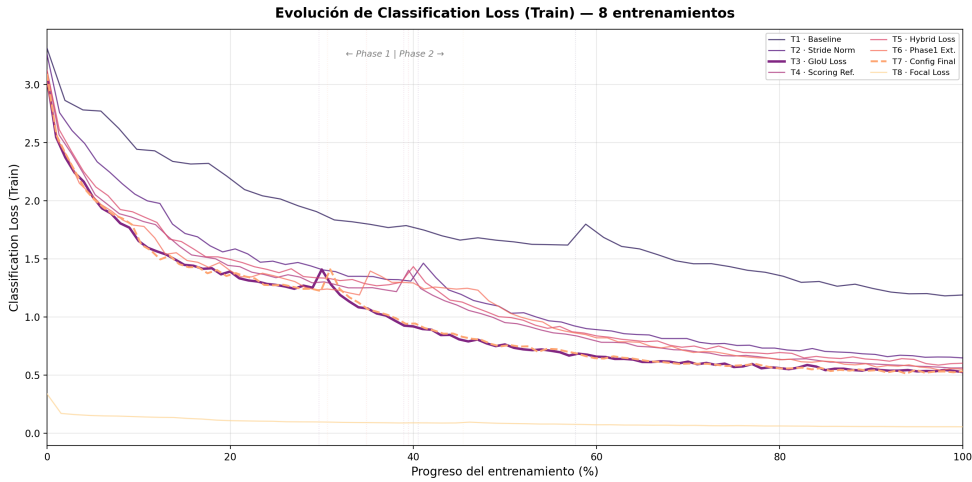
**Tabla 13.** Resumen Entrenamientos Modelo 1 – MobileNetV3+FCOS

| Métrica (Test)    | Train 1  | Train 2  | Train 3       | Train 4  | Train 5  | Train 6  | Train 7       | Train 8       |
|-------------------|----------|----------|---------------|----------|----------|----------|---------------|---------------|
| <b>mAP@50</b>     | 0.4304   | 0.5600   | 0.5675        | 0.5936   | 0.5887   | 0.5572   | <b>0.6120</b> | 0.5954        |
| <b>mAP@50-95</b>  | N/C      | N/C      | 0.2602        | 0.2644   | 0.2703   | 0.2511   | <b>0.2824</b> | 0.2615        |
| <b>Precision</b>  | 0.5427   | 0.6049   | <b>0.6609</b> | 0.3462   | 0.3505   | 0.3290   | 0.3716        | 0.2071        |
| <b>Recall</b>     | 0.5291   | 0.6271   | 0.6276        | 0.6886   | 0.6845   | 0.6558   | 0.6872        | <b>0.7143</b> |
| <b>F1-Score</b>   | 0.5358   | 0.6158   | <b>0.6438</b> | 0.4607   | 0.4636   | 0.4382   | 0.4824        | 0.3211        |
| <b>Épocas</b>     | 52       | 74       | 101           | 77       | 76       | 86       | 98            | 66            |
| <b>Tiempo</b>     | 12.1 min | 15.9 min | 23.6 min      | 17.9 min | 17.9 min | 19.5 min | ~23 min       | 17.0 min      |
| <b>Inferencia</b> | 5.0 ms   | 4.6 ms   | 4.8 ms        | 5.4 ms   | 4.9 ms   | 5.1 ms   | 5.0 ms        | 5.9 ms        |

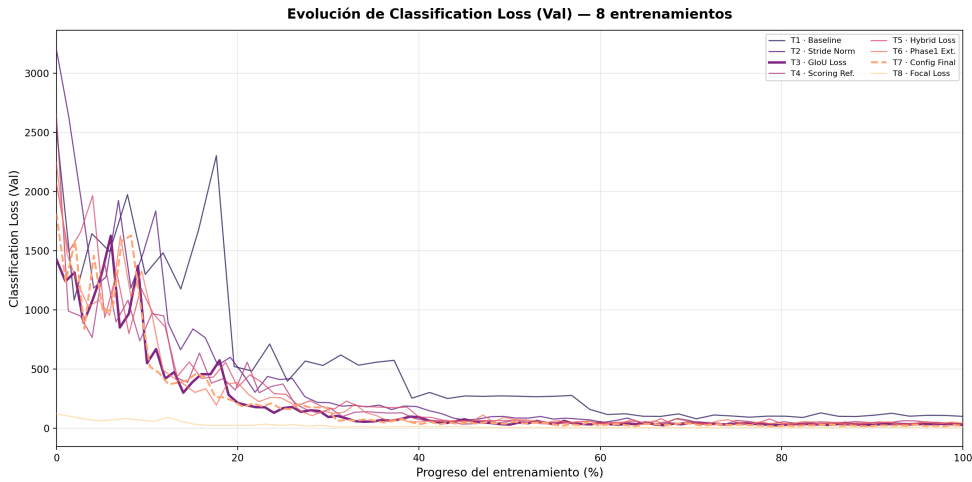
**Tabla 14.** *mAP@50 por clase en Modelo 1 – MobileNetV3+FCOS*

| Clase        | Train 1      | Train 2      | Train 3      | Train 4      | Train 5      | Train 6      | Train 7      | Train 8      |
|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| dog          | 0.406        | 0.463        | 0.496        | 0.502        | <b>0.522</b> | 0.442        | 0.506        | 0.435        |
| door         | 0.339        | 0.519        | 0.503        | 0.533        | 0.544        | 0.464        | <b>0.569</b> | 0.619        |
| obstacle     | 0.302        | 0.405        | 0.458        | 0.512        | 0.464        | 0.460        | 0.523        | <b>0.526</b> |
| person       | 0.521        | 0.636        | 0.636        | 0.682        | 0.666        | 0.683        | <b>0.691</b> | 0.686        |
| stair        | 0.584        | <b>0.777</b> | 0.745        | 0.739        | 0.747        | 0.736        | 0.770        | 0.712        |
| <b>Media</b> | <b>0.430</b> | <b>0.560</b> | <b>0.568</b> | <b>0.594</b> | <b>0.589</b> | <b>0.557</b> | <b>0.612</b> | <b>0.595</b> |

**Figura 36.** *Evolución de la pérdida de clasificación en Modelo 1 – MobileNetV3+FCOS*

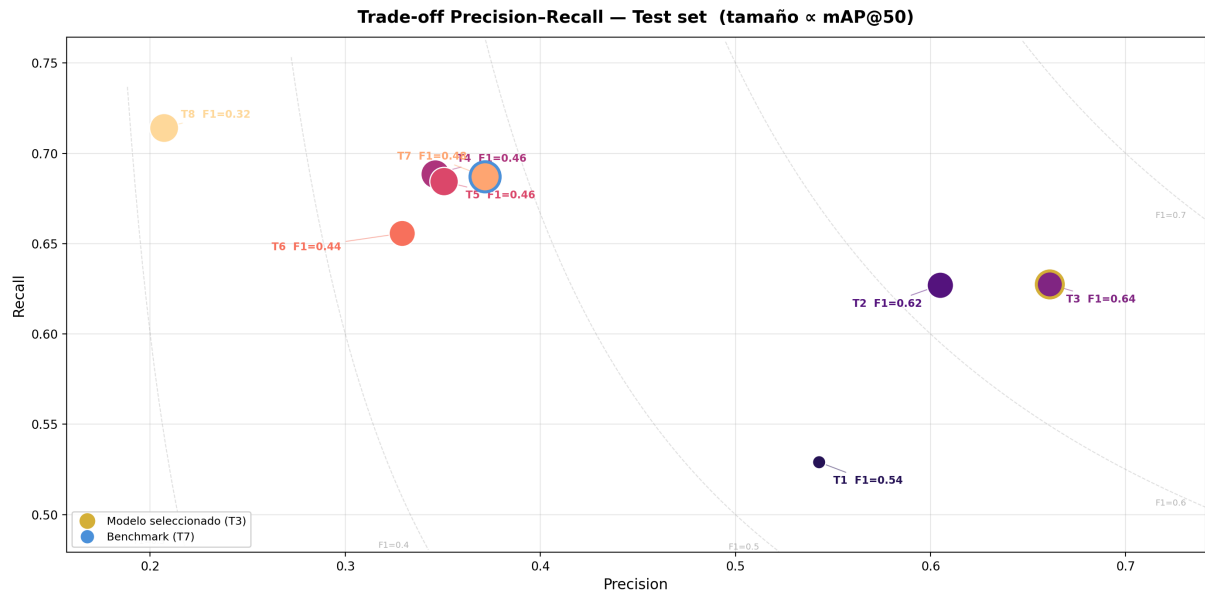


**(a) Entrenamiento**

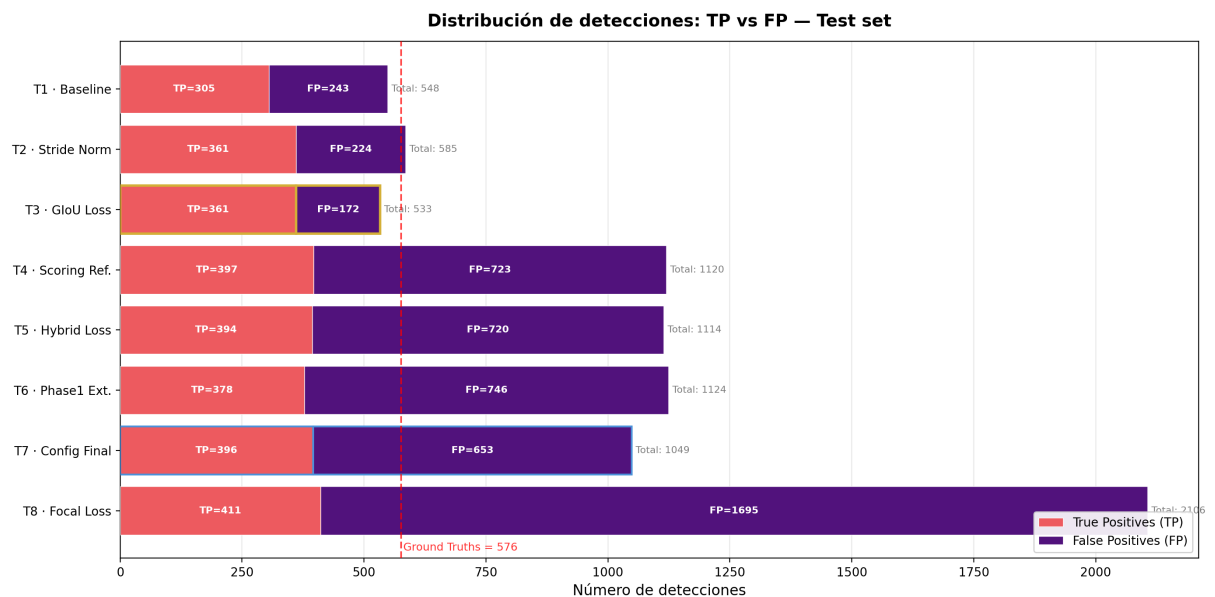


**(b) Validación**

**Figura 37. Trade-off Precisión-Recall Test Set en Modelo 1 – MobileNetV3+FCOS**



**Figura 38. Distribución de Detecciones en Modelo 1 – MobileNetV3+FCOS**

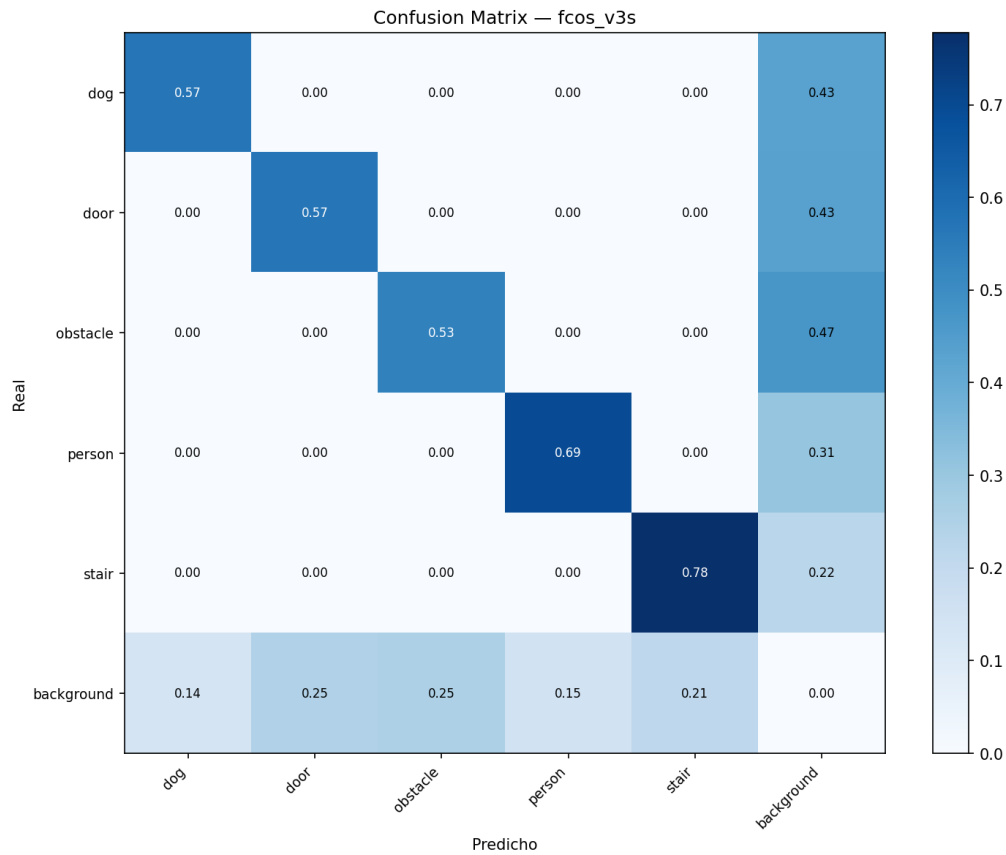


A partir de las Figuras 36, 37 y 38, se concluye que, a pesar de que el modelo *Train 7* obtuvo el *mAP* más alto, su alta tasa de falsos positivos lo hace ineficiente. En LCMR, un falso positivo (p. ej., detectar un obstáculo inexistente) puede causar comportamientos erráticos o frenados innecesarios. Por ello, el *Train 3* se identifica como el modelo superior para producción debido a su Precisión de 0.66 y su bajo ratio de falsas alarmas. Mediante un barrido de umbrales (*Threshold Sweep*), se determinó que elevar el umbral de confianza (*conf\_threshold*) de 0.25



a 0.40 en el modelo *Train 3* optimiza el desempeño a partir de una reducción de FP en -24.3%, mejora de F1 relativo en +7.6% y pérdida de Recall de solo -3.5%.

**Figura 39.** Matriz de Confusión para Modelo 1 – MobileNetV3+FCOS T3



### 5.3.2. Modelo 2: Arquitectura YOLO

La arquitectura de la familia YOLO se basa en la categoría Nano, lo que implica una reducción agresiva en la profundidad y anchura de las capas para permitir su ejecución en microcontroladores. Posee aproximadamente  $2.6 \times 10^6$  parámetros en su versión estándar. En la versión optimizada con remoción de DFL, este número se reduce a 2,505,750 parámetros. Aunque el entrenamiento se realiza típicamente a 640 píxeles, la arquitectura ha sido adaptada para una resolución de exportación de 224x224 píxeles.

El diseño compone tres bloques fundamentales:

- **Backbone:** Hereda la eficiencia de YOLO11n, utilizando bloques de convolución optimizados para la extracción de características multiescala

- **Neck:** La red opera con tres escalas de salida para detectar objetos de diferentes tamaños. P3 (Stride 8): Resolución de 28×28 para objetos pequeños. P4 (Stride 16): Resolución de 14×14 para objetos medianos. P5 (Stride 32): Resolución de 7×7 para objetos grandes.
- **Head:** La modificación más significativa entre YOLO11n y YOLO26 es la transición hacia el desuso de Distribution Focal Loss (FL) y una salida de regresión directa. En la arquitectura estándar (YOLO11n), la rama de regresión de cajas utiliza un  $reg\_max = 16$ , lo que genera 64 canales probabilísticos (16 bins por cada lado de la caja). Esto requiere una operación de softmax seguida de una suma ponderada (integral) para obtener las coordenadas finales. En la configuración (YOLO26), se modificó la arquitectura para establecer un  $reg\_max = 1$ . Los canales de salida de la rama cv2[0] pasan de 64 a solo 4. El modelo entrega distancias directas, eliminando la necesidad de cálculos complejos en el firmware del ESP32-S3. Esto simplifica el post-procesamiento a una simple conversión de distancias a coordenadas de caja delimitadora.

Se realizaron tres iteraciones principales (denominadas Train 1, 2 y 3), cada una diseñada para validar una hipótesis de optimización específica.

- **Train 1 - Línea Base (*Baseline*):** El primer experimento utilizó pesos preentrenados de YOLO11n y el optimizador por defecto (AdamW). Obteniendo como resultado que el modelo alcanzó un  $mAP50$  de 0.7544 en el conjunto de test. Se observó que la Fase 2 no superó a la Fase 1 en términos de precisión, lo que sugirió que el optimizador AdamW fue demasiado agresivo para el ajuste fino o que el ritmo de aprendizaje no fue el óptimo.
- **Train 2 - Optimización mediante MuSGD:** Se sustituyó el optimizador por *MuSGD* (una combinación de SGD y el algoritmo *Muon* para la ortogonalización de matrices de pesos). Se ajustó el LR de la Fase 2 a 0.0005 y se redujo el umbral de confianza a 0.15 para favorecer el *Recall*. Se obtuvo el mejor desempeño global con un  $mAP50$  de 0.7747 y un F1-score de 0.7517. *MuSGD* demostró una mejor convergencia y estabilidad. A pesar de ser un 25% más lento en tiempo de cómputo, la mejora en la calidad de las cajas delimitadoras (IoU) justificó su uso.

- **Train 3 - Eliminación de Pérdida de Distribución Focal (DFL) y Supresión No Máxima con YOLO26 (NMS):** Con el objetivo de simplificar el post-procesamiento en el ESP32-S3 y aprovechando las características clave de la nueva versión de esta familia de modelos, YOLO26, se modificó la arquitectura para eliminar el módulo DFL y el post-procesamiento separado NMS, pasando de 64 canales probabilísticos en la salida de regresión a 4 canales directos de distancia (*reg\_max=1*). El *mAP50* descendió levemente a 0.7466 (-3.6% respecto a Train 2). Sin embargo, tiene la ventaja de reducir el tamaño del modelo y simplificar drásticamente el firmware del microcontrolador al no requerir la operación *softmax integral* para obtener las coordenadas.

A continuación, se presentan en las Tabla 15 y 16 la comparativa de las métricas obtenidas en el conjunto de test para las tres configuraciones de *YOLO Family*. La distribución de detecciones refuerza la robustez de T2 frente a sus contrapartes a partir de la Efectividad; T2 logró 400 Verdaderos Positivos (TP), acercándose significativamente al total de *Ground Truths* (437), mientras mantuvo el nivel más bajo de Falsos Positivos (FP = 169) de toda la serie. El Train 3, a pesar de utilizar pesos nativos yolo26n.pt, presentó la mayor tasa de error con 203 FP, lo que se traduce en una menor precisión (0.7853).

**Tabla 15.** Resumen Entrenamientos Modelo 2 – YOLO Family

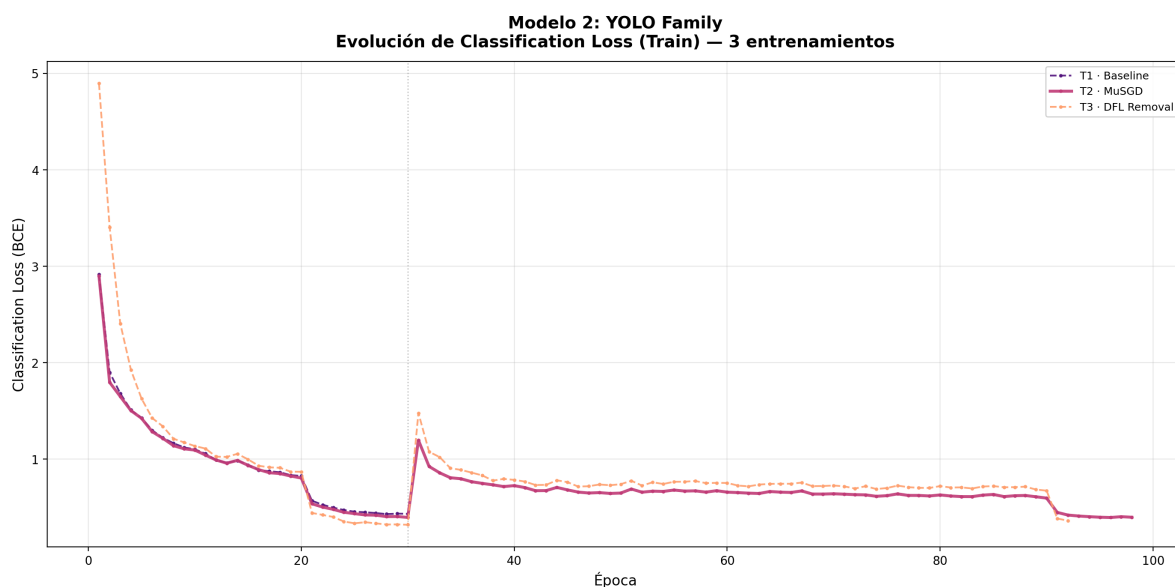
| Métrica (Test)             | Train 1     | Train 2       | Train 3             |
|----------------------------|-------------|---------------|---------------------|
| <b>mAP@50</b>              | 0.7544      | <b>0.7747</b> | 0.7466              |
| <b>mAP@50-95</b>           | 0.5153      | <b>0.5456</b> | 0.5333              |
| <b>Precision</b>           | 0.8264      | <b>0.8324</b> | 0.7853              |
| <b>Recall</b>              | 0.6402      | <b>0.6853</b> | 0.6522              |
| <b>F1-Score</b>            | 0.7215      | <b>0.7517</b> | 0.7126              |
| <b>Épocas (P1+P2)</b>      | 100 (30+70) | 98 (30+68)    | 92 (30+62)          |
| <b>Optimizador</b>         | AdamW       | MuSGD         | MuSGD               |
| <b>Pesos Preentrenados</b> | yolo11n.pt  | yolo11n.pt    | <b>yolo26n.pt</b>   |
| <b>reg_max / DFL</b>       | 16 / DFL    | 16 / DFL      | <b>1 / Identity</b> |
| <b>Parámetros</b>          | 2,590,815   | 2,590,815     | <b>2,505,750</b>    |
| <b>Umbral de Confianza</b> | 0.25        | 0.15          | 0.15                |
| <b>Tiempo</b>              | 26.0 min    | 32.6 min      | 30.5 min            |
| <b>Inferencia</b>          | 2.6 ms      | 2.9 ms        | 4.0 ms              |

**Tabla 16.** *mAP@50 por clase en Modelo 2 – YOLO Family*

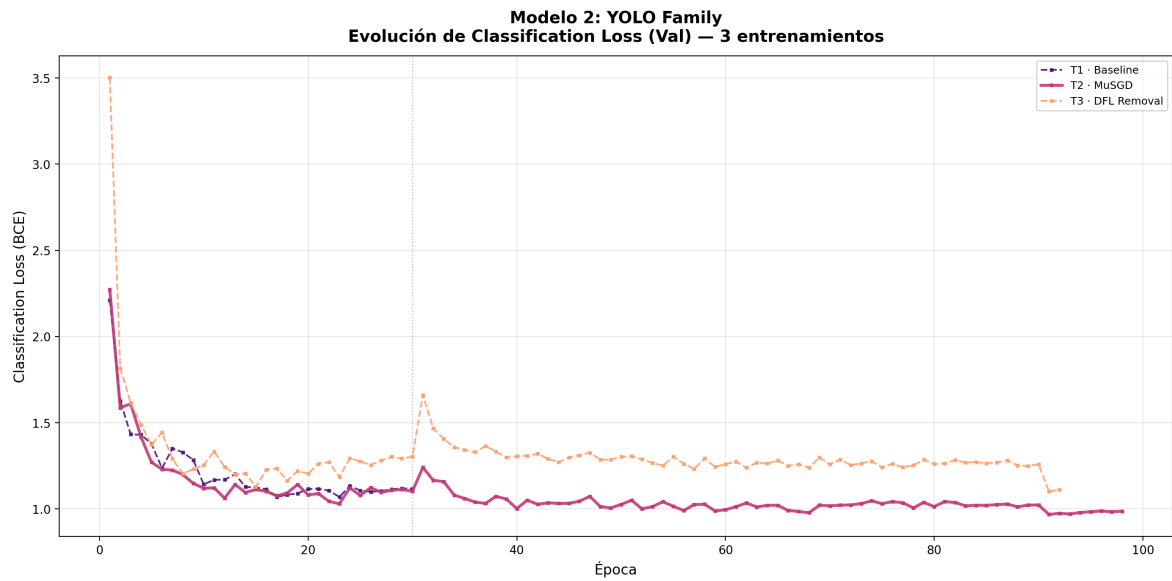
| Clase           | Train 1       | Train 2       | Train 3       |
|-----------------|---------------|---------------|---------------|
| <b>dog</b>      | 0.7740        | 0.7956        | <b>0.8047</b> |
| <b>door</b>     | 0.6531        | <b>0.6601</b> | 0.6606        |
| <b>obstacle</b> | 0.7011        | <b>0.7384</b> | 0.6543        |
| <b>person</b>   | 0.8301        | 0.8661        | <b>0.8860</b> |
| <b>stair</b>    | <b>0.8138</b> | 0.8133        | 0.7272        |
| <b>Media</b>    | <b>0.7544</b> | <b>0.7747</b> | <b>0.7466</b> |

La evaluación granular por categoría (Tabla 16) permite identificar la especialización y las limitaciones de la arquitectura en el dataset IODC YOLO. La clase dominante (*person*) presenta el mejor desempeño en los tres modelos, alcanzando un pico de *mAP50* de 0.8860 en T3. Esto sugiere que las características morfológicas de los humanos son capturadas con alta fidelidad por la arquitectura YOLO26. Por su parte, la clase crítica (*dog*) tiene una mejora progresiva, pasando de 0.7740 (T1) a 0.8047 (T3). El reto de navegación para un LCMR está en la distinción de las clases *door* y *obstacle*; estas presentan los valores más bajos y estables (aprox. 0.66 y 0.65-0.73 respectivamente). La variabilidad geométrica de los obstáculos y la similitud de las puertas con las paredes del fondo representan el mayor reto para estos modelos.

**Figura 40.** *Evolución de la pérdida de clasificación en Modelo 2 – YOLO Family*

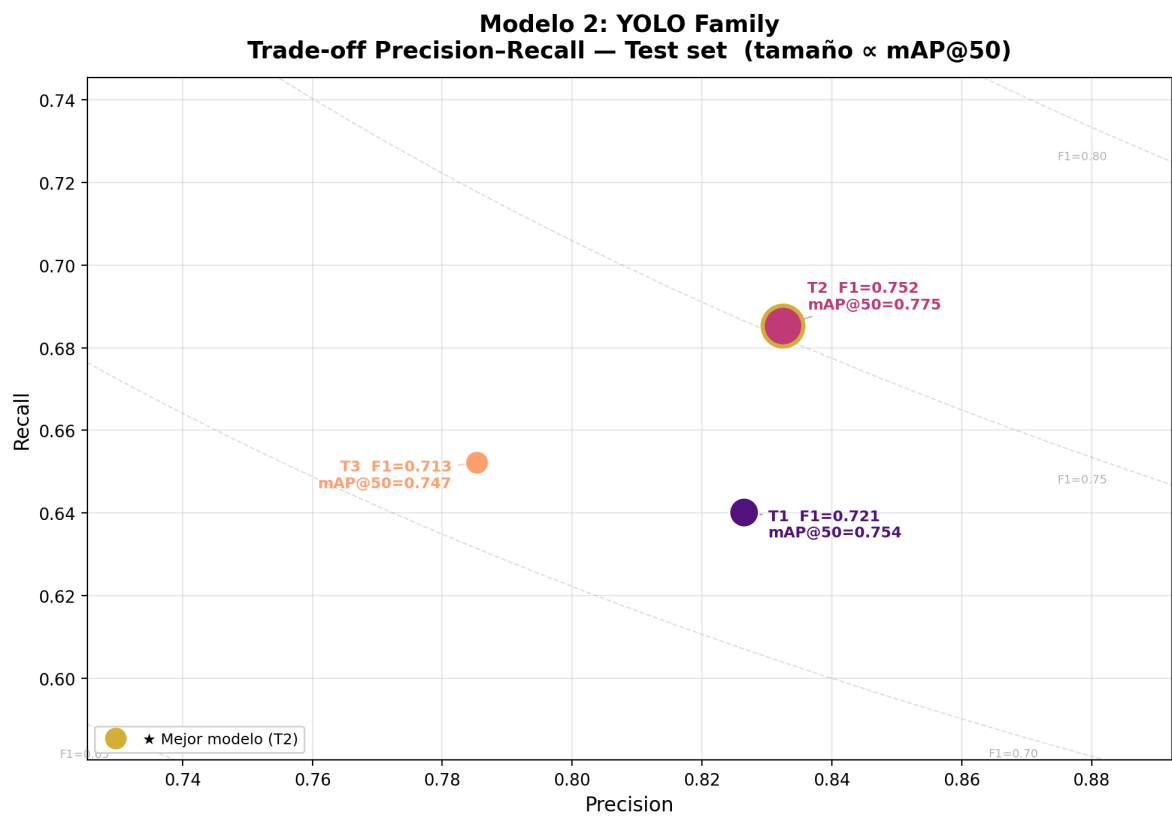


(a) Entrenamiento

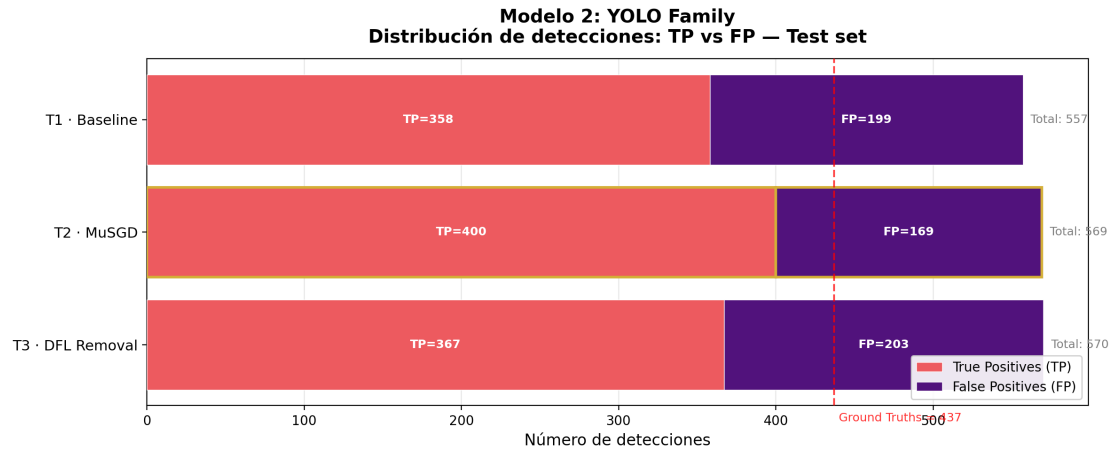


(b) Validación

**Figura 41. Trade-off Precisión-Recall Test Set en Modelo 2 – YOLO Family**

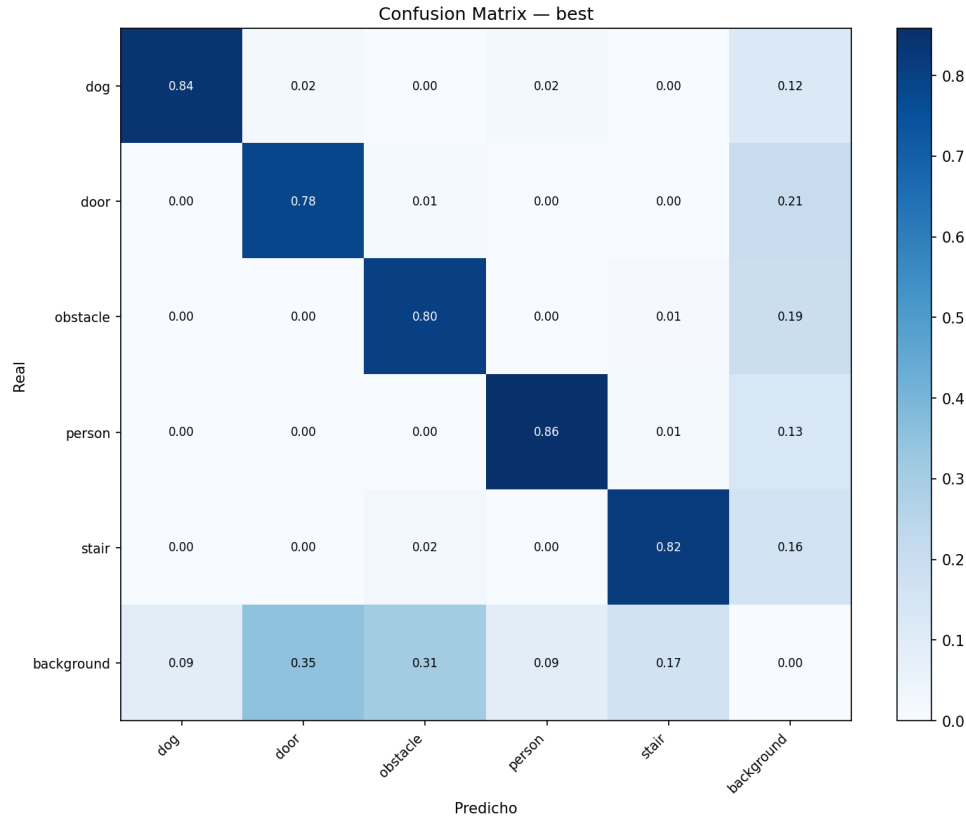


**Figura 42. Distribución de Detecciones en Modelo 2 – YOLO Family**



Los resultados demuestran que la configuración *Train 2 (MuSGD + yolo11n.pt)* proporciona la solución más equilibrada para la detección de objetos, con una precisión superior al 83% y la mejor capacidad de recuperación de objetos reales (Recall). Sin embargo, el modelo *Train 3* ofrece una alternativa arquitectónica simplificada que, a pesar de una ligera caída en el mAP global, podría facilitar la integración en sistemas de tiempo real debido a su menor conteo de parámetros y estructura de salida directa.

**Figura 43. Matriz de Confusión para Modelo 2 seleccionado – YOLO26 T2**



### 5.3.3. Modelo 3: Arquitectura ESPDet-Pico Espressif

La arquitectura de esta familia de modelos iteró a través de cuatro ciclos de entrenamiento con variaciones estructurales, consolidándose en la reimplementación fiel de la arquitectura oficial del repositorio *espressif/esp-detection*. El diseño se caracteriza por ser un detector anchor-free de ultra-bajo peso ( $\sim 0.36M$  de parámetros), optimizado para el despliegue en microcontroladores ESP32-S3 con una configuración de escala  $n$  que define un  $depth=0.50$  y un  $width=0.25$ . El diseño estructural se compone tres bloques fundamentales:

- **Backbone (Extracción de características):** Se compone de una topología oficial de 11 capas diseñada para maximizar la extracción de características con un coste computacional mínimo. Utiliza una secuencia de bloques especializados que incluyen *DSCConv* (convoluciones separables en profundidad), *ESPBlockLite*, *DSC3k2* y módulos *SCDown* para la reducción de resolución. Además, incorpora un módulo *SPPF* (*Spatial Pyramid Pooling - Fast*) para capturar contextos a múltiples escalas y una convolución final de *kernel*  $7 \times 7$ . Genera tres mapas de características principales con *strides* de  $[8, 16, 32]$ , denominados P3/8, P4/16 y P5/32, con profundidades de hasta 128 canales en el nivel más profundo.
- **Neck (Fusión Multiescala mediante FPN Bi-direccional):** Actúa como un puente integrador que combina la información semántica profunda con la información espacial de las capas iniciales mediante un diseño *Top-down + Bottom-up FPN*. Emplea operaciones de *Upsampling* y concatenación seguidas de bloques *ESPBlock* para propagar la información semántica hacia las resoluciones más altas. Refuerza la localización mediante convoluciones de *stride* 2 y concatenaciones ascendentes, permitiendo que las características de baja escala se enriquezcan con el contexto global. Produce tres niveles de salida refinados con canales configurados específicamente (32 para P3, 128 para P4 y 128 para P5) para alimentar la cabeza de detección.
- **Head (Cabezal de Detección):** Implementa una estrategia desacoplada que separa la tarea de localización de la de clasificación, eliminando la necesidad de anchor boxes predefinidos. La regresión utiliza dos capas consecutivas de *DSCConv* seguidas de una *Conv2d* con  $reg\_max = 1$ . Esta configuración permite la regresión directa de las 4 distancias (l,t,r,b) desde el centro del objeto hacia los bordes, optimizando la

capacidad del modelo para dispositivos embebidos. La clasificación está compuesta por bloques repetidos de *DWConv* + *Conv* que finalizan en una capa *Conv2d(nc)* para predecir la probabilidad de las clases. El modelo exporta los resultados en un formato *interleaved* (cajas y puntuaciones entrelazadas por nivel), facilitando su procesamiento directo mediante la librería *esp-ppq* hacia el formato nativo *.espd* del ESP32-S3.

El **Train 1**, inició con una arquitectura simplificada denominada v1 (*Custom*), compuesta por apenas 22.8K parámetros. El objetivo era evaluar la viabilidad de un modelo de extrema ligereza con un *backbone* basado en convoluciones separables y un *FPN* simplificado. Sin embargo, los resultados de esta fase inicial evidenciaron un fracaso funcional. Con un *mAP@50* de apenas 0.0105, el modelo mostró una incapacidad crítica para distinguir las 5 clases del dataset IODC (*dog, door, obstacle, person, stair*). El análisis diagnóstico determinó que la capacidad representacional era insuficiente para un problema multi-clase, y la ausencia de pesos preentrenados impidió la convergencia en un espacio de búsqueda tan restringido.

**Tabla 17.** *Resumen Entrenamientos Modelo 3 – ESPDet Expressif*

| Métrica (Test)             | Train 1              | Train 2           | Train 3                          | Train 4                                   |
|----------------------------|----------------------|-------------------|----------------------------------|-------------------------------------------|
| <b>mAP@50</b>              | 0.0105               | <b>0.6203</b>     | 0.5993                           | 0.6052                                    |
| <b>mAP@50-95</b>           | 0.0023               | <b>0.3078</b>     | 0.3220                           | 0.2701                                    |
| <b>Precision</b>           | 0.0009               | 0.2956            | 0.2156                           | <b>0.3298</b>                             |
| <b>Recall</b>              | 0.1597               | 0.7235            | <b>0.7663</b>                    | 0.7278                                    |
| <b>F1-Score</b>            | 0.0017               | 0.4197            | 0.3365                           | <b>0.4539</b>                             |
| <b>Épocas (P1+P2)</b>      | 120 (40+80)          | 113 (50+63)       | 131 (50+81)                      | <b>94 (50+44)</b>                         |
| <b>Loss</b>                | BCE                  | BCE               | Focal (gamma=2.0,<br>alpha=0.25) | <b>BCE</b>                                |
| <b>Optimizador</b>         | AdamW                | AdamW             | AdamW                            | AdamW                                     |
| <b>Umbral de Confianza</b> | 0.25                 | 0.25              | 0.25                             | <b>0.35</b>                               |
| <b>Umbral de Oclusión</b>  | 0.45                 | 0.45              | 0.45                             | <b>0.40</b>                               |
| <b>Tiempo</b>              | 23.6 min             | 21.7 min          | 27.0 min                         | <b>20.2 min</b>                           |
| <b>Inferencia (GPU)</b>    | 34.4 ms              | 4.6 ms            | 3.6 ms                           | 5.0 ms                                    |
| <b>Tamaño ONNX</b>         | 0.109 MB             | 1.41 MB           | 1.41 MB                          | 1.41 MB                                   |
| <b>Arquitectura</b>        | Custom v1<br>(22.8K) | Oficial v2 (361K) | Oficial v2 + Focal (361K)        | <b>Oficial v2 + NMS<br/>tuning (361K)</b> |



Tras los resultados del *baseline*, el **Train 2** procedió a implementar la arquitectura Oficial de *Espressif* (v2), elevando el número de parámetros a aproximadamente 361K (0.36M). Se aplicó una estrategia de *Transfer Learning* utilizando pesos preentrenados disponibles en la tarea de detección (cat-detection), cargando exitosamente el 99.97% de los grupos de parámetros. Como se observa en la Tabla 17, el salto cualitativo fue sustancial. La adopción de la arquitectura oficial permitió alcanzar un mAP@50 del 62.03%, validando la arquitectura de *Espressif* como la base mínima viable para el proyecto.

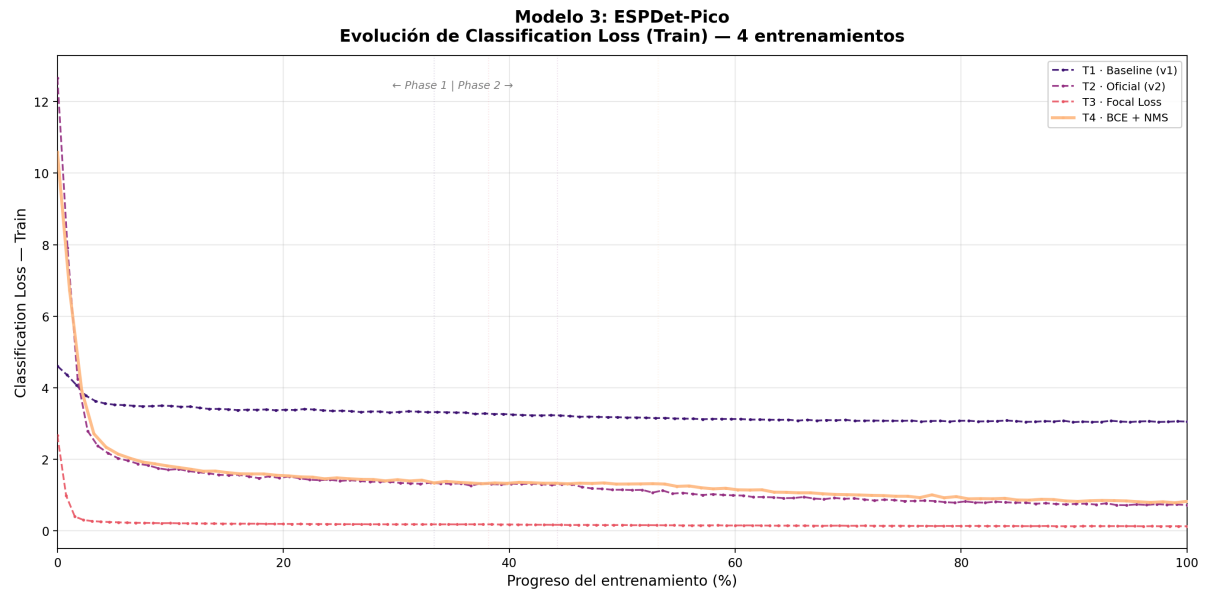
Con una base funcional establecida, el **Train 3** buscó resolver el principal cuello de botella del modelo: la baja precisión debida a una alta tasa de falsos positivos (FP). Se planteó la hipótesis que el reemplazo de la función de pérdida *Binary Cross Entropy* (BCE) por *Sigmoid Focal Loss* ( $\gamma=2.0, \alpha=0.25$ ) mejoraría la supresión del fondo (background). Los resultados, no obstante, refutaron la hipótesis. El uso de *Focal Loss* en un modelo de solo 0.36M parámetros provocó una degradación del 27.1% en la precisión y un aumento del 88.7% en los falsos positivos totales. Se concluyó que la reducción de gradiente en negativos fáciles priva a estos micro-modelos de la señal necesaria para aprender una supresión de fondo efectiva.

**Tabla 18.** mAP@50 por clase en Modelo 3 – ESPDet *Espressif*

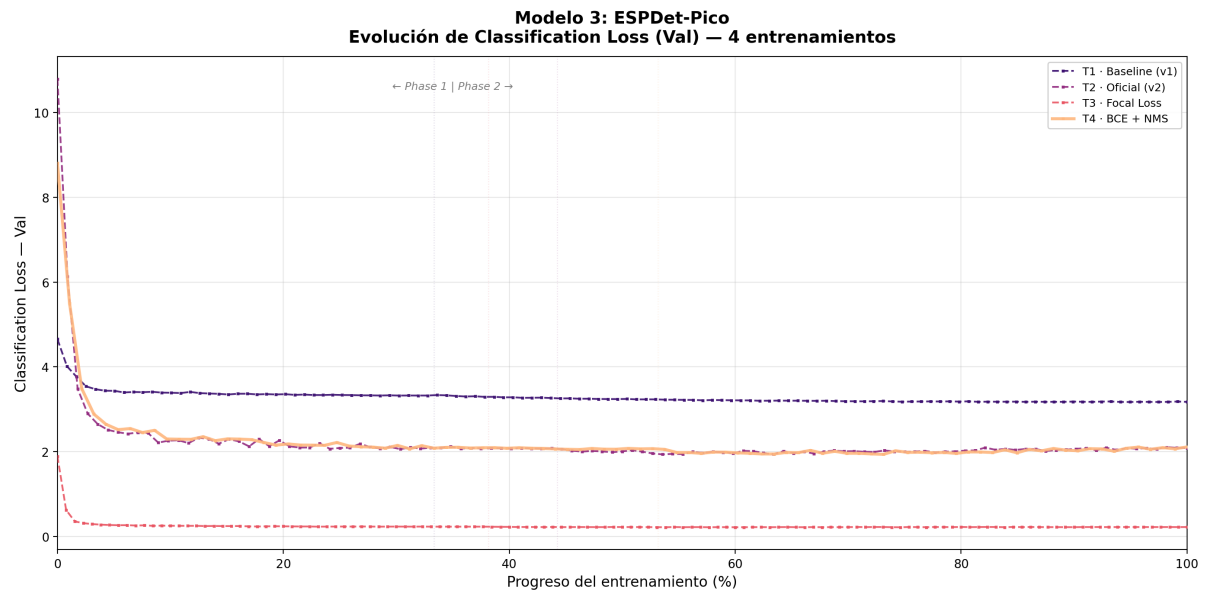
| Clase           | Train 1       | Train 2       | Train 3       | Train 4       |
|-----------------|---------------|---------------|---------------|---------------|
| <b>dog</b>      | 0.0000        | 0.6085        | 0.5989        | 0.5305        |
| <b>door</b>     | 0.0763        | 0.5846        | 0.5589        | 0.5572        |
| <b>obstacle</b> | 0.0000        | 0.5043        | 0.4418        | 0.4618        |
| <b>person</b>   | 0.0000        | 0.6742        | 0.7078        | <b>0.7059</b> |
| <b>stair</b>    | 0.0004        | 0.7299        | 0.6893        | <b>0.7708</b> |
| <b>Media</b>    | <b>0.0383</b> | <b>0.6203</b> | <b>0.5993</b> | <b>0.6052</b> |

En la cuarta iteración, el **Train 4**, se revirtió a la pérdida BCE y se ajustaron los umbrales de inferencia (*NMS Tuning*): el umbral de confianza (*conf\_threshold*) se elevó de 0.25 a 0.35 y el de *IoU* se ajustó a 0.40. Esta estrategia resultó ser la más exitosa de la serie, con *F1-Score máximo* alcanzando 0.4539, un incremento del 8.1% respecto a Train 2; reducción de Falsos Positivos (FP) de toda la serie (875 frente a los 1670 de la versión con *Focal Loss*); y preservación de Recall, al mantener el 72.78%, factor crítico para la seguridad de un LCMR.

**Figura 44. Evolución de la pérdida de clasificación en Modelo 3 – ESPDet Expressif**



(a) Entrenamiento

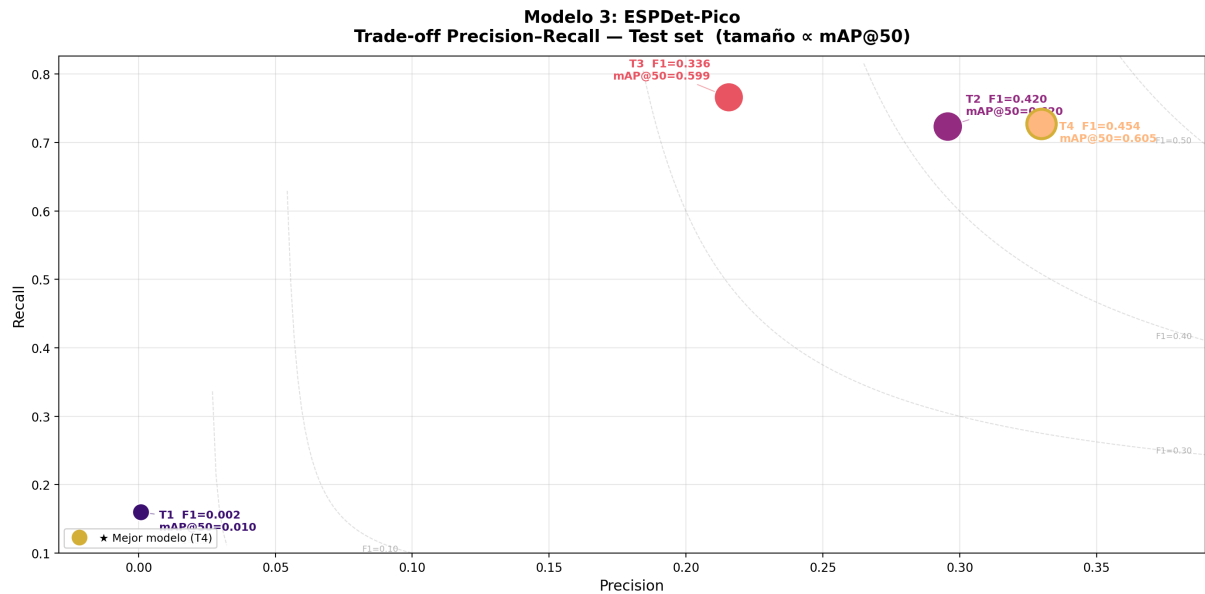


(b) Validación

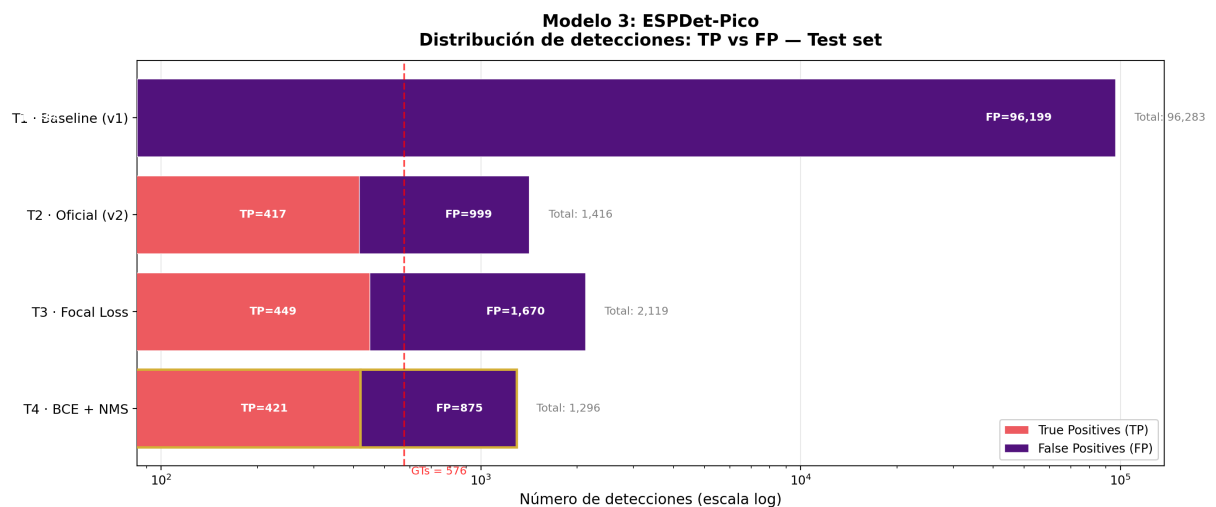
El análisis del equilibrio entre precisión y exhaustividad (*Precision-Recall trade-off*), de la Figura 45, permite identificar visualmente al modelo Train 4 (BCE + NMS) como la configuración más equilibrada. En el gráfico de burbujas, se observa que T4 alcanza el F1-Score más alto (0.454), situándose en una región de compromiso óptimo donde se maximiza la precisión sin sacrificar drásticamente el recall. Por el contrario, T3 (Focal Loss), a pesar de

lograr el recall más elevado de la serie (0.766), se desplaza hacia la izquierda en el eje de precisión, lo que confirma una sensibilidad excesiva ante el ruido del fondo.

**Figura 45. Trade-off Precisión-Recall Test Set en Modelo 3 – ESPDet Expressif**

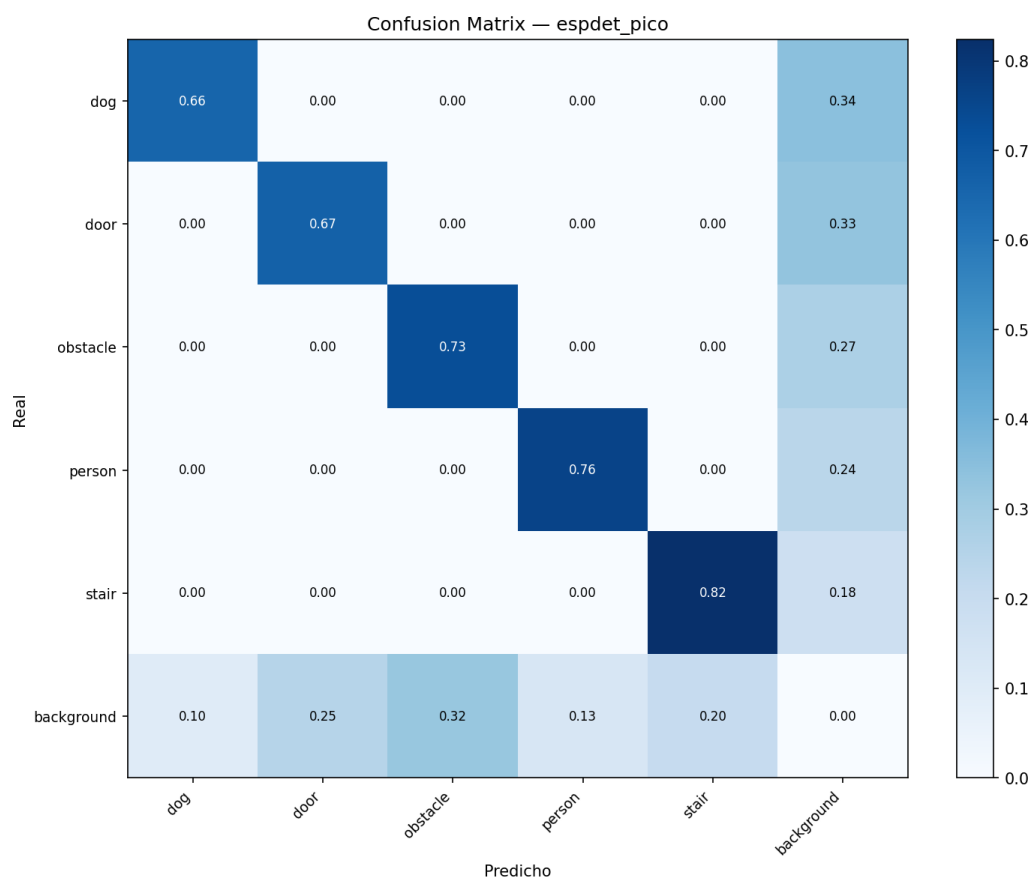


**Figura 46. Distribución de Detecciones en Modelo 3 – ESPDet Expressif**



Finalmente, la distribución de detecciones, mostrada en la Figura 44 en escala logarítmica, ilustra la eficiencia del *NMS Tuning* aplicado en T4. A diferencia de T1, que genera un volumen masivo e inoperable de falsos positivos (96,199 FP), T4 logra la supresión de fondo más efectiva de la serie oficial con 875 FP, manteniendo un nivel de verdaderos positivos (421 TP) comparable al de T2. Este comportamiento valida la selección de T4 como el modelo con mejor capacidad de discriminación, permitiendo un despliegue en el ESP32-S3 que minimiza las alertas erróneas en tareas de asistencia visual.

**Figura 47. Matriz de Confusión para Modelo 3 seleccionado – ESPDet T4**



## 5.4. Cuantización y Adaptación para TinyML

Esta sección detalla el proceso de optimización y adaptación de los modelos de aprendizaje profundo para su despliegue en sistemas embebidos de recursos limitados, específicamente para el microcontrolador ESP32-S3. El objetivo principal es transformar modelos entrenados en punto flotante (FP32) a formatos compatibles con el acelerador de hardware ESP-DL, en formato entero (INT8), garantizando que el tamaño binario se mantenga por debajo del límite crítico de 3.0 MB.

### 5.4.1. Fundamentos de la Cuantización Post-Entrenamiento (PTQ)

La cuantización es una técnica esencial en el paradigma de TinyML que permite reducir la precisión de los pesos y activaciones de una red neuronal, pasando de representaciones de 32 bits (FP32) a enteros de 8 bits (INT8). Para este trabajo, se ha implementado una Cuantización Post-Entrenamiento (PTQ, por sus siglas en inglés) simétrica, la cual mapea los valores

originales al rango  $[-128, +127]$  mediante un factor de escala calculado durante una fase de calibración estática. La operación de cuantización se define matemáticamente por la Ecuación 4. Donde  $s$  representa el factor de escala. En el formato optimizado de Espressif (ESPDL), este factor de escala se codifica como una potencia de dos ( $s = 2^{\text{exponent}}$ ), permitiendo que las operaciones de re-escalado se ejecuten eficientemente mediante desplazamientos de bits (*bit-shifts*) en el acelerador de hardware, evitando multiplicaciones costosas en punto flotante.

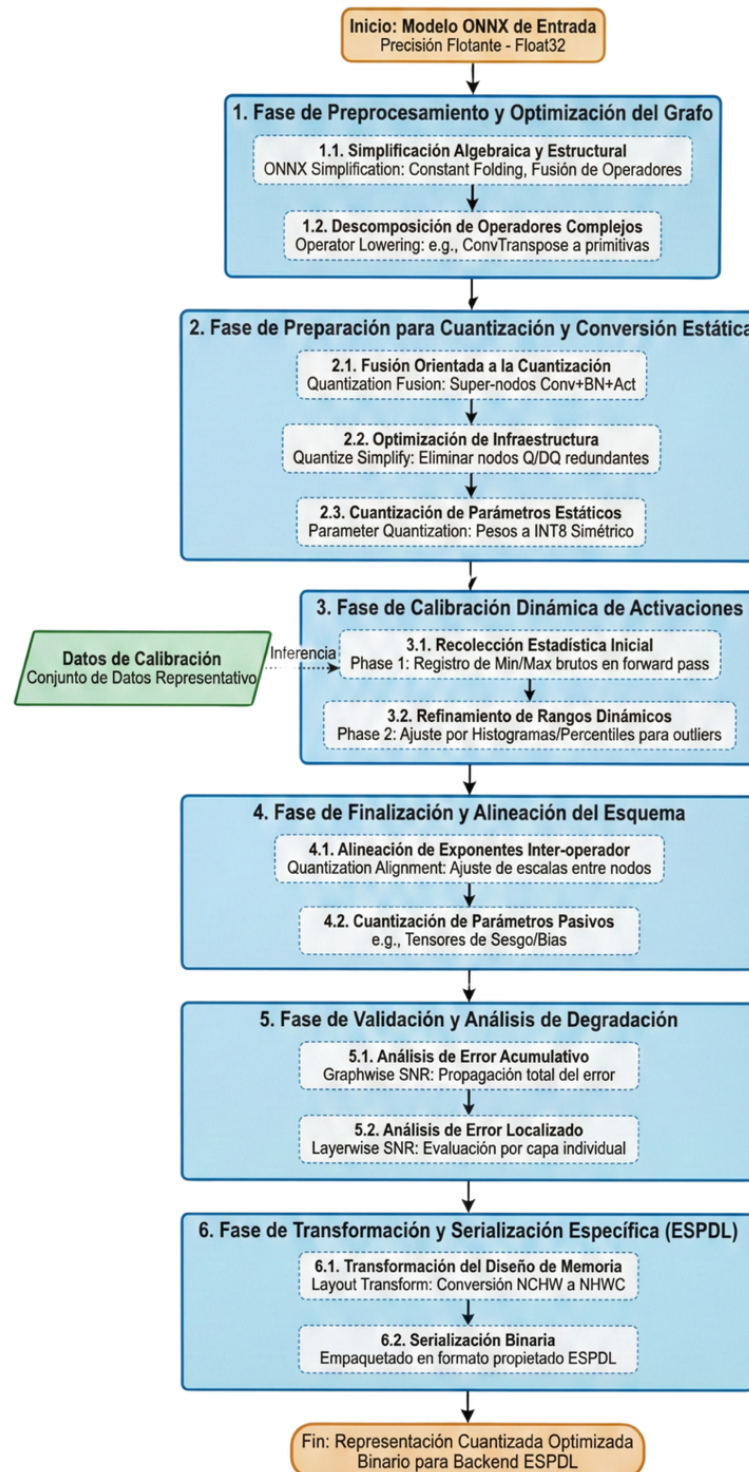
$$Q(x) = \text{clamp}\left(\text{round}\left(\frac{x}{s}\right), -128, 127\right) \quad \text{Ecuación (4)}$$

#### 5.4.2. Flujo de Conversión de Formatos

El proceso de despliegue sigue una trayectoria de transformación multi-etapa desde el entorno de entrenamiento en la nube hasta el firmware del dispositivo final:

- **Exportación ONNX:** Los modelos entrenados en PyTorch se exportan al formato interoperable ONNX preservando la precisión FP32.
- **Preprocesamiento y Corrección:** Ciertas arquitecturas, como YOLO26, requieren el ajuste de atributos de ejes negativos (ej., *axis=-1*) a valores positivos para asegurar la compatibilidad con las transformaciones de *layout* NCHW a NHWC requeridas por el hardware objetivo.
- **Cuantización con esp-ppq:** Se emplea la herramienta *esp-ppq v1.2.6* para ejecutar la calibración estática basada en un dataset representativo (500 imágenes del *split* de entrenamiento IODC). Este proceso genera el archivo binario *.espd* que contiene los pesos cuantizados, las tablas de búsqueda (LUT) para activaciones no lineales y la topología del grafo optimizada. El pipeline interno creado en *espd\_quantize\_onnx()* ejecuta las fases mostradas en la Figura 45.

**Figura 48. Pipeline de Cuantización**



Los modelos seleccionados para cuantización corresponden a las mejores versiones de cada familia, según los criterios documentados en sus respectivos registros de entrenamiento:

- **FCOS T3:** Mejor F1 (0.6438) + mejor Precision (0.6609) de la serie. Referencia operativa para despliegue. *Post-sweep: conf\_threshold* óptimo = 0.40.
- **YOLO26 T3:** Mejor mAP@50 (0.7747) y F1 (0.7517) de la serie. *Optimizer MuSGD* vs *baseline AdamW*.
- **ESPDet T4:** Mejor F1 (0.4539) y Precision (0.3298) con *NMS tuning* (conf=0.35, IoU=0.40). Arquitectura oficial Espressif, candidato principal ESP32-S3.

#### 5.4.3. Validación de Restricciones de Memoria y Tamaño Binario

Uno de los requisitos técnicos más estrictos del proyecto es que el modelo no exceda los 3.0 MB de ocupación en la memoria Flash, compartida con el firmware del sistema. Los resultados de la cuantización demuestran una reducción significativa del tamaño, logrando factores de compresión de hasta 3.66×, como se observa en la Tabla 19.

**Tabla 19.** Modelos seleccionados para cuantización

| Métrica               | FCOS T3           | YOLO26 T3             | ESPDet T4                             |
|-----------------------|-------------------|-----------------------|---------------------------------------|
| Input Shape           | [1, 3, 224, 224]  | [1, 3, 224, 224]      | [1, 3, 224, 224]                      |
| Channel Format        | NCHW              | NCHW                  | NCHW                                  |
| Outputs               | 2 (boxes, scores) | 1 tensor [1, 9, 1029] | 6 interleaved (box0, score0, box1...) |
| ONNX (FP32)           | 4.75 MB           | 9.95 MB               | 1.42 MB                               |
| Parámetros            | 1.23M             | ~2.6M                 | ~361K                                 |
| mAP@50 (test, FP32)   | 0.5675            | 0.7747                | 0.6052                                |
| F1-Score (test, FP32) | 0.6438            | 0.7517                | 0.4539                                |
| Fix ONNX requerido    | No                | Sí (ejes negativos)   | No                                    |

#### 5.4.4. Análisis de Desempeño y Degradación

A pesar de cumplir con las restricciones de tamaño, la cuantización introduce ruido que puede degradar el desempeño métrico de los modelos, medido a través del *Noise:Signal Power Ratio* (NSPR) métrica de error de cuantización expresada como porcentaje y la precisión media (mAP@50). Se realizó una validación funcional comparando la ejecución FP32 frente a la simulación INT8 (utilizando el motor *TorchExecutor* de *esp-ppq*) sobre el conjunto de test, apoyadas de las comparaciones visuales de la inferencia de cada modelo en su formato FP32 vs INT8 que se observan en las Figuras 44, 45 y 46, se pueden extraer las siguientes conclusiones:

- **ESPDet T4:** Presentó la mayor robustez, con una degradación del mAP@50 de solo el 11.1% (0.5985 a 0.5319). Su diseño nativo para el ecosistema Espressif evitó errores de exponente y facilitó una transición eficiente a aritmética entera.
- **YOLO26 T3:** Inicialmente, este modelo sufrió un colapso (0 detecciones) debido a la salida unificada del formato ONNX estándar. Tras una re-exportación con salidas separadas para cajas y puntuaciones, se logró rescatar el modelo, alcanzando un mAP@50 INT8 de 0.4343 (degradación del 18.0%).
- **FCOS T3:** Se determinó que es arquitectónicamente incompatible con la cuantización INT8 en este hardware. La presencia de capas de *Instance Normalization* y canales iniciales estrechos en el *backbone MobileNetV3-Small* provocan una propagación de error que silencia los *logits* de clasificación, resultando en una degradación del 99.9%.

**Tabla 20.** Comparativa resultados de cuantización

| Métrica                    | FCOS T3 |              | YOLO26 T3 |          | ESPDet T4 |         |
|----------------------------|---------|--------------|-----------|----------|-----------|---------|
|                            | FP32    | INT8         | FP32      | INT8     | FP32      | INT8    |
| <b>mAP@50</b>              | 0.5878  | 0.0003       | 0.5297    | 0.4343   | 0.5985    | 0.5319  |
| <b>Degradación mAP@50</b>  | -       | 99,9%        | -         | 18.0%    | -         | 11.1%   |
| <b>mAP@50-95</b>           | 0.2673  | 0.0001       | 0.3272    | 0.2740   | 0.2655    | 0.2419  |
| <b>Precision</b>           | 0.4320  | 0.0087       | 0.8114    | 0.8195   | 0.3708    | 0.3360  |
| <b>Recall</b>              | 0.6639  | 0.0015       | 0.5748    | 0.4749   | 0.7108    | 0.6655  |
| <b>F1</b>                  | 0.5234  | 0.0025       | 0.6729    | 0.6013   | 0.4874    | 0.4465  |
| <b>Detecciones totales</b> | 865     | 23           | 385       | 308      | 1116      | 1181    |
| <b>Peso (ONNX/ESPD)</b>    | 4.75 MB | 1.74 MB      | 9.95 MB   | 2.72 MB  | 1.42 MB   | 0.52 MB |
| <b>Compresión</b>          | -       | 2.74x        | -         | 3.66x    | -         | 2.72x   |
| <b>Tiempo Conversión</b>   | -       | 169.4 s      | -         | 78.8 s   | -         | 83.7 s  |
| <b>Viabilidad</b>          | -       | Incompatible | -         | Ajustado | -         | Viable  |

Se concluye que ESPDet T4 es el candidato óptimo para el despliegue primario, dada su extrema ligereza (0.52 MB) y baja degradación métrica. YOLO26 T2 se mantiene como una alternativa viable de mayor complejidad, siempre que se utilice el formato de exportación de salidas independientes para preservar la resolución de los scores de clase. El modelo FCOS T3 queda formalmente descartado para ejecución en INT8 dentro de esta plataforma.



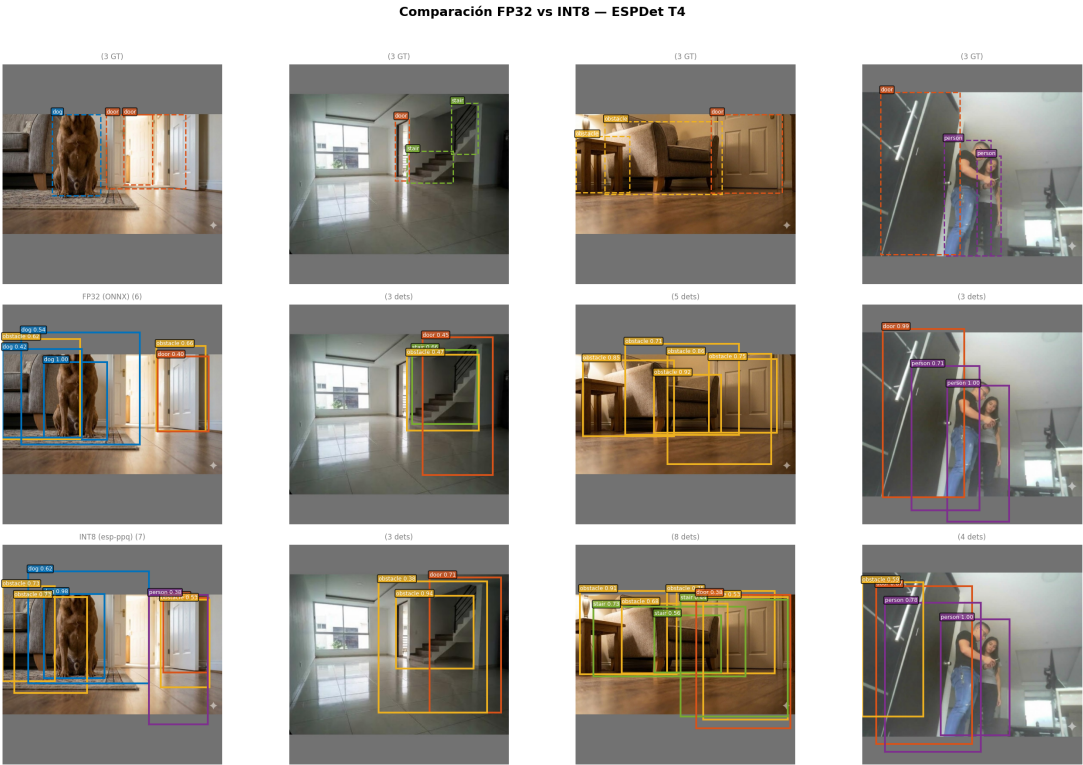
**Figura 49. Comparación FP32 vs INT8 Modelo 1 – FCOS T3**



**Figura 50. Comparación FP32 vs INT8 Modelo 2 – YOLO26 T3**



**Figura 51. Comparación FP32 vs INT8 Modelo 3 - ESPDet T4**



5.5.Síntesis

a